ML Systems
(*Efficient LLM Inference on GPUs*)

Introduction

Monsoon, 2026

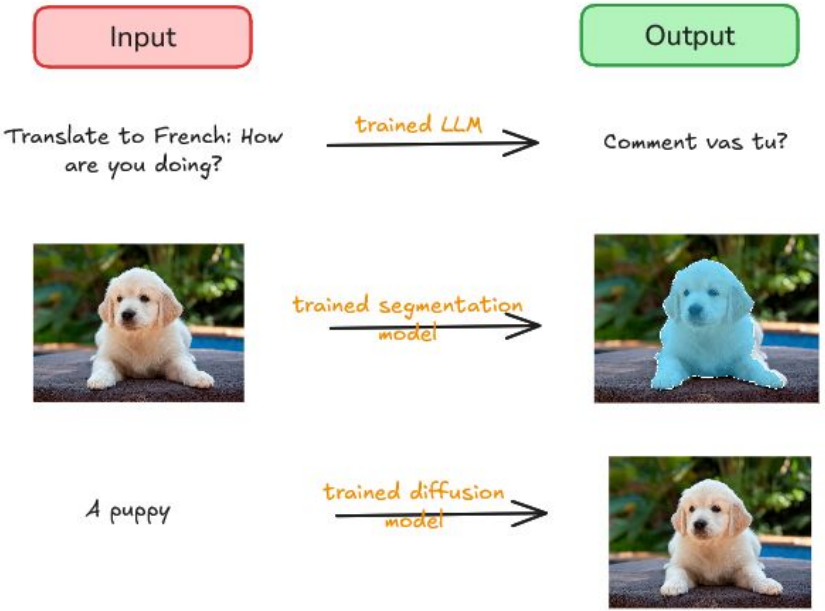
Pankaj Pansari
Plaksha University

AI4103: Efficient LLM Inference on GPUs

- Quantitative, systems-oriented deep dive into LLM inference
- Focused on transformer-architecture
- Primary platform: NVIDIA GPUs and CUDA
- Not a broad survey of every part of ML systems

ML Inference

The process of giving a trained ML model an input and obtaining output



Serving: software system that accepts requests, schedules and does inference for concurrent users

Pervasive Use of Generative AI

Google's token volume per month:

9.7T -> 480T -> 3.2Q

Think of token volume considering all providers!

Source: Google I/O, 2026

Inference Scaling

A task is no longer necessarily one prompt and one response

Longer (reasoning models): one call generates more reasoning tokens

Wider (test-time scaling): several candidate calls are sampled and verified

Deeper (agentic): an agent repeatedly acts, observes and adapts

Challenges of LLM Inference

Token-by-token generation: Inference is harder to parallelize than training

Two-phases: Inference has two phases with very different computational characteristics

Tight latency requirements: Reasoning and agentic models demand fast token generation

Long context: Puts more demand on constrained GPU memory

Large models: Splitting model on multiple GPUs introduces communication costs

A Puzzle

- Let's give a prompt of 128 tokens on a Qwen3-8B model running on A100 GPU
- We measured 34.86ms as time to generate a single token
- We did 2 x 8.2 x 10e9 FLOPs = 16.4 GFLOPs
- A100 can do 312 TFLOPs/sec (denoted by TFLOPS)
- Compute efficiency = 0.151%

A Puzzle (Details)

- Let's give a prompt of 128 tokens on a Qwen3-8B (actually has 8.2B params) model running on A100 GPU (40 GB HBM, 1555 GB/s peak bandwidth)
- We measured 34.86ms as time to generate a single token (median over 128 generated tokens)
- We did 2*8.2*10e9 FLOPs = 16.4 GFLOPs (we'll derive the 2 * num_params thumb rule for inference FLOPs later in course)
- A100 can do 312 TFLOPs/sec (denoted by TFLOPS) (tensor cores, dense matmul)
- Compute efficiency = 0.151%
- Achieved bandwidth = ~470 GB/s = 30.2 % of peak
- What accounts for the gap?

Efficiency is the Key

- GPUs are expensive and power-hungry
- Compute efficiency dictates cost, power consumption, hardware costs, responsiveness
- Becomes more critical at larger scale

Goal: *Methods for Efficient Deployment of LLMs*

ML Systems lets us predict performance, measure it, and reason about improvements.

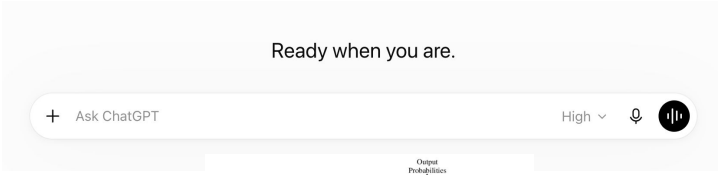
How to make a chat assistant

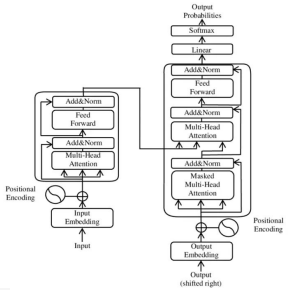
Application

Model + workload

Software

Hardware

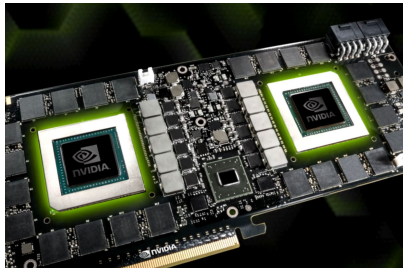




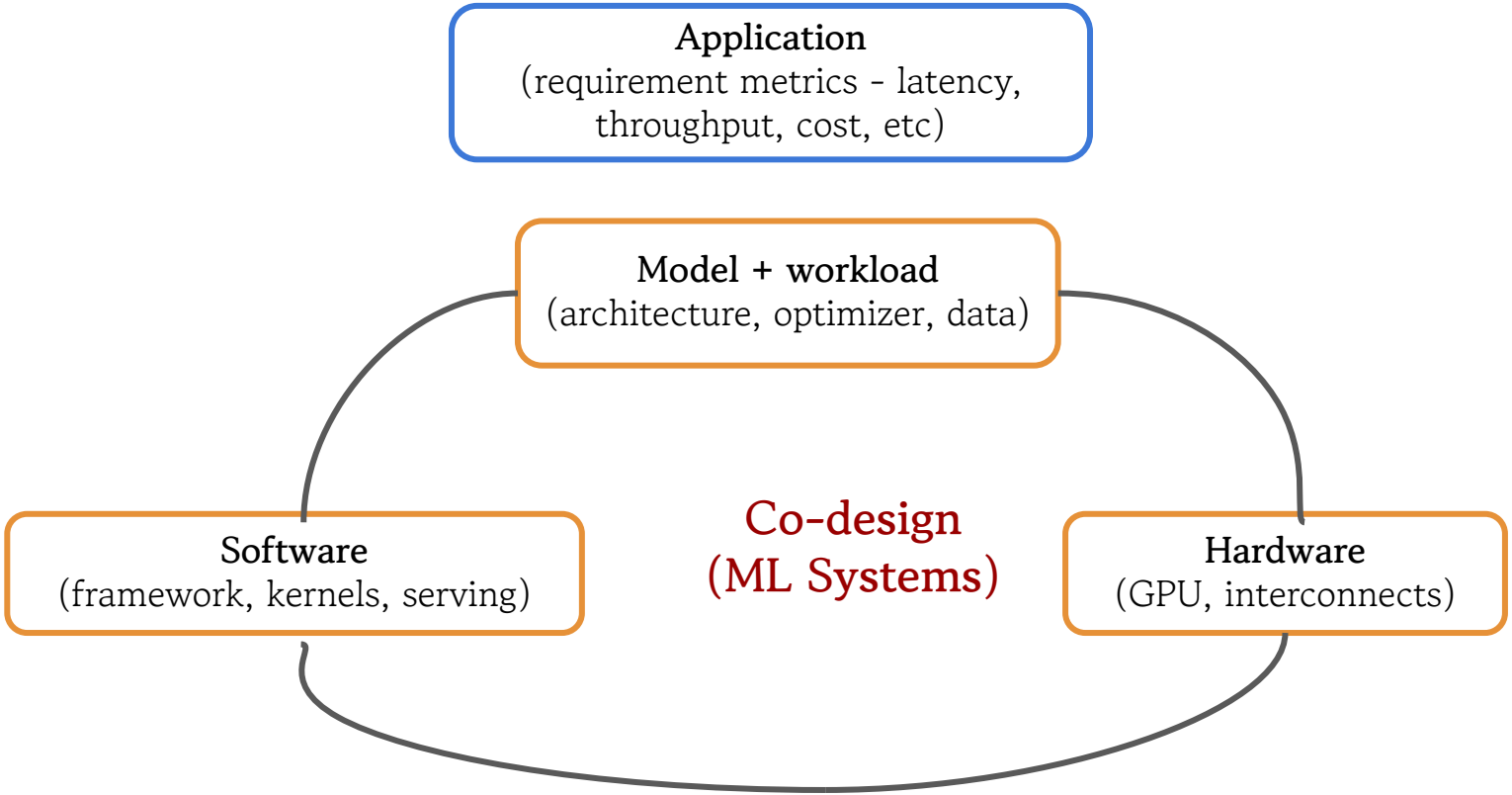
PyTorch

vLLM
Easy, fast, and cheap LLM serving for everyone

NVIDIA CUDA



How to make an efficient chat assistant



What is ML Systems?

Co-design of model architecture, software stack, and hardware

- How to design models that run efficiently on hardware?
- How to write software to run a model efficiently on hardware?

Why Study ML Systems?

- **Architecture co-design:** ML Systems work decides what models are possible in practice
  - FlashAttention -> avoids materializing the quadratic attention matrix -> long context -> agentic workloads
  - Mixture-of-expert models (MoEs): expert-parallel kernels
  - Continuous batching + KV paging -> higher concurrency and better serving efficiency

- **Principled nature:** One can make an estimate about performance, measure using profiler, reason about improvements that could work. Rare in ML.

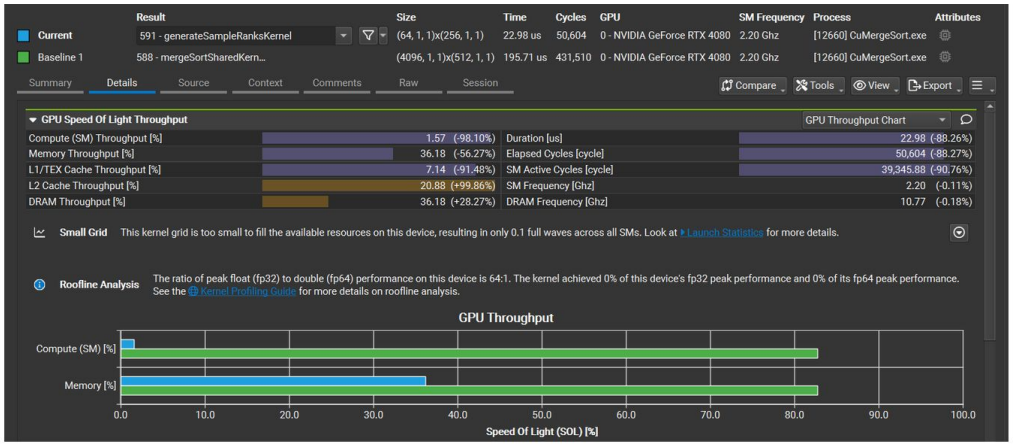
Why Study ML Systems? (contd.)

- **Durability:** The concepts are more durable. The AI models and hardware are rapidly evolving but at systems level, the principles for running them efficiently on GPUs very stable

- **Efficiency:** Can we minimize the hardware cost and requirements of running capable models?

What you'll be able to do

- Account for FLOPs and memory traffic of transformer inference
- Use roofline analysis and profiling tools to identify performance bottlenecks
- Write and optimize basic CUDA kernels
- Predict latency and throughput of prefill and decode workloads
- Compare batching, scheduling, cache-management, and multi-GPU strategies



Screenshot: Nsight Compute profiler

Scope of AI4103

- Transformer-based LLMs
  - Dominant models
  - Autoregressive nature poses special computational challenges
- LLM inference only
  - Significant percentage of AI computational workload
  - Narrow enough to analyze deeply in one course, still involving kernels, memory management, scheduling, and distributed execution
- Primary platform: Nvidia GPUs and CUDA
  - Still the most common AI accelerators; others like AMD are catching up
  - We'll briefly touch upon TPU architecture

Course Overview

- **Performance reasoning:** transformer accounting, arithmetic intensity, roofline
- **GPU execution:** architecture, CUDA, matmul, profiling
- **Single-GPU inference:** prefill, decode, KV cache, batching, FlashAttention
- **Scaling serving:** scheduling, cache management, multi-GPU inference, MoE

Grading

- 2 programming assignments: 50% (25% each)
- 2 class-tests: 20% (10% each)
- End-sem: 30%

In-class Policy

To make the classroom work well for everyone:

- Please arrive on time and minimize entering or leaving during the lecture
- Keep phones away during class
- Use laptops only when requested for an activity

Prerequisites

- Comfortable programming in Python and C++ in Linux
- Familiarity with the transformer forward pass
- Basic systems concepts: processes, threads, caches, virtual memory, and concurrency
- No prior CUDA programming required

Compute Resources

- Rs 2k compute credit per student on 

- Best to go for least expensive GPU that satisfies the memory requirement
- Assignments involve CUDA kernels and LLM inference profiling; no training involved

- By the end of the course, you'll be able to explain and improve the 34.9 ms/token result
- **Next lecture**: counting FLOPs and bytes for transformer layer during inference